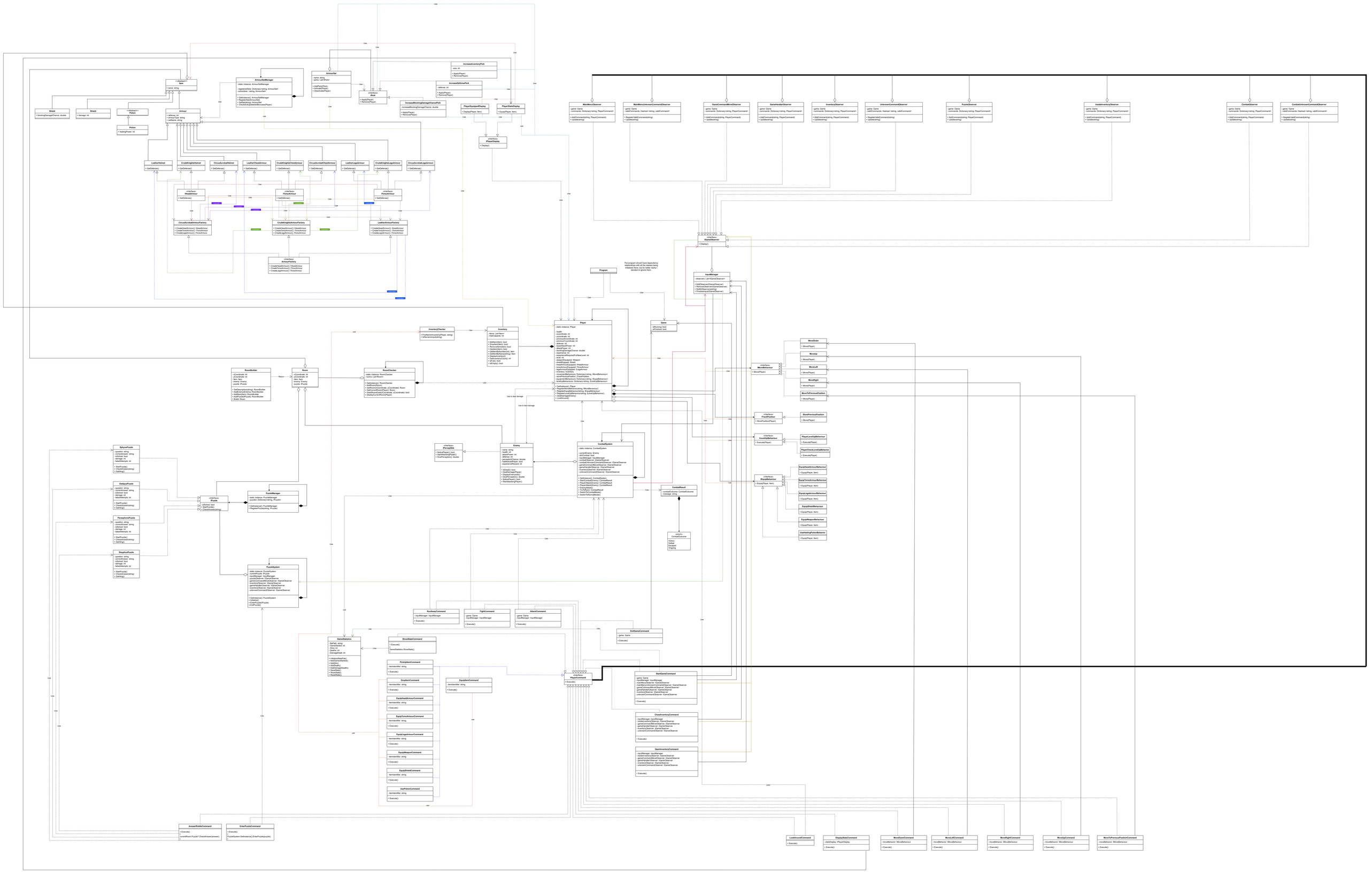
Shield
+ blockingDamageChance: double
Shield
+ damage: int
<<Abstract>>
Potion
Potion
+healingPower: int
<<Abstract>>
Item
+ name: string
Armour
+ defense: int
+ armourType: string
+ setName: string
ArmourSetManager
- static instance: ArmourSetManager
- registeredSets: Dictionary<string, ArmourSet>
- activeSets: <string, ArmourSet>
+ GetInstance(): ArmourSetManager
+ RegisterSet(ArmourSet set)
+ GetSet(string): ArmourSet
+ CheckAndUpdateSetBonuses(Player)
ArmourSet
- name: string
- perks: List<Perk>
+ AddPerk(Perk)
+ ActivatePlayer()
+ DeactivatePlayer()
<<Interface>>
IPerk
+ Apply(Player)
+ Remove(Player)
IncreaseInventoryPerk
- size: int
+ Apply(Player)
+ Remove(Player)
IncreaseDefensePerk
- defense: int
+ Apply(Player)
+ Remove(Player)
IncreaseBlockingDamageChancePerk
- increaseBlockingDamageChance: double
+ Apply(Player)
+ Remove(Player)
PlayerEquipped Display
+ Display(Player, Item)
PlayerStatsDisplay
+ Display(Player, Item)
<<Interface>>
IPlayerDisplay
+ Display()
LeatherHelmet
+ GetDefense()
CrudeKnightsHelmet
+ GetDefense()
CircusAcrobatHelmet
+ GetDefense()
LeatherChestArmour
+ GetDefense()
CrudeKnightsChestArmour
+ GetDefense()
CircusAcrobatChestArmour
+ GetDefense()
LeatherLegsArmour
+ GetDefense()
CrudeKnightsLegsArmour
+ GetDefense()
CircusAcrobatLegsArmour
+ GetDefense()
<<Interface>>
IHeadArmour
+ GetDefense()
<<Interface>>
ITorsoArmour
+ GetDefense()
<<Interface>>
ILegsArmour
+ GetDefense()
CircusAcrobatArmourFactory
+ CreateHeadArmour(): IHeadArmour
+ CreateTorsoArmour(): ITorsoArmour
+ CreateLegsArmour(): ITorsoArmour
CrudeKnightsArmourFactory
+ CreateHeadArmour(): IHeadArmour
+ CreateTorsoArmour(): ITorsoArmour
+ CreateLegsArmour(): ITorsoArmour
LeatherArmourFactory
+ CreateHeadArmour(): IHeadArmour
+ CreateTorsoArmour(): ITorsoArmour
+ CreateLegsArmour(): ITorsoArmour
<<Interface>>
ArmourFactory
+ CreateHeadArmour(): IHeadArmour
+ CreateTorsoArmour(): ITorsoArmour
+ CreateLegsArmour(): ITorsoArmour
MainMenuObserver
- game: Game
- commands: Dictionary<string, PlayerCommand>
+ AddCommand(string, PlayerCommand)
+ Update(string)
MainMenuUnknownCommandObserver
- game: Game
- validCommands: Hashset <string, validCommand>
+ RegisterValidCommand(string)
+ Update(string)
GameCommandMoveObserver
- game: Game
- commands: Dictionary<string, PlayerCommand>
+ AddCommand(string, PlayerCommand)
+ Update(string)
GameHotkeyObserver
- game: Game
- commands: Dictionary<string, PlayerCommand>
+ AddCommand(string, PlayerCommand)
+ Update(string)
InventoryObserver
- game: Game
- commands: Dictionary<string, PlayerCommand>
+ AddCommand(string, PlayerCommand)
+ Update(string)
UnknownCommandObserver
- game: Game
- validCommands: Hashset <string, validCommand>
+ RegisterValidCommand(string)
+ Update(string)
PuzzleObserver
- game: Game
- commands: Dictionary<string, PlayerCommand>
+ AddCommand(string, PlayerCommand)
+ Update(string)
InsideInventoryObserver
- game: Game
- commands: Dictionary<string, PlayerCommand>
+ AddCommand(string, PlayerCommand)
+ Update(string)
CombatObserver
- game: Game
- commands: Dictionary<string, PlayerCommand>
+ AddCommand(string, PlayerCommand)
+ Update(string)
CombatUnknownCommandObserver
- game: Game
- validCommands: Hashset <string, validCommand>
+ RegisterValidCommand(string)
+ Update(string)
<<Interface>>
IGameObserver
+ Display()
InputManager
-observers: List<IGameObserver>
+ AddObserver(IGameObserver)
+ RemoveObserver(IGameObserver)
+ ClearObservers()
+ ProcessInput(IGameObserver)
Program
The program should have dependency on Game, but it's not actually initialized there, but for better clarity I decided to ignore them.
Player
- static instance: Player
- health
- xcoordinate: int
- ycoordinate: int
- previousXcoordinate: int
- previousYcoordinate: int
- defense: int
- baseMaxHPower: int
- maxHPower: int
- blockingDamageChance: double
- experience: int
- experiencePointsForNextLevel: int
- level: int
- weaponEquipped: IWeapon
- shieldEquipped: IShield
- headArmourEquipped: IHeadArmour
- torsoArmourEquipped: ITorsoArmour
- legsArmourEquipped: ILegsArmour
- inventory: Inventory
- activeBehaviors: Dictionary<string, IMoveBehaviour>
- storePreviousPosition: ITrackPosition
- levelUpBehaviors: Dictionary<string, ILevelUpBehaviour>
- levelUpBehaviors: Dictionary<string, IEquipBehaviour>
+ GetInstance(): Player
+ RegisterMoveBehaviour(string, IMoveBehaviour)
+ RegisterEquipBehaviour(string, IEquipBehaviour)
+ RegisterLevelUpBehaviour(string, ILevelUpBehaviour)
+ DealDamage(Enemy)
+ LookAround()
Use to deal damage
Use to deal damage
Game
- isRunning: bool
- isFinished: bool
InventoryChecker
+ FindItemInInventory(Player, string)
+ IsItemInInventory(string)
Inventory
- items: List<Item>
- maxCapacity: int
+ AddItem(Item): bool
+ DropItem(Item): bool
+ RemoveItem(Item): bool
+ HasItem(Item): bool
+ GetItemByIndex(int): Item
+ GetItemByName(string): Item
+ ClearInventory()
+ GetInventoryCount(): int
+ IsFull(): bool
+ IsEmpty(): bool
RoomBuilder
- xCoordinate: int
- yCoordinate: int
- item: Item
- enemy: Enemy
- puzzle: IPuzzle
+ SetDescription(string): RoomBuilder
+ AddItem(Item): RoomBuilder
+ AddEnemy(Enemy): RoomBuilder
+ AddPuzzle(IPuzzle): RoomBuilder
+ Build: Room
Return
Room
xCoordinate: int
yCoordinate: int
item: Item
enemy: Enemy
puzzle: IPuzzle
RoomChecker
- static instance: RoomChecker
- rooms: List<Room>
+ GetInstance(): RoomChecker
+ AddRoom(Room)
+ GetRoomCoordinate_xCoordinate(): Room
+ GetCurrentRoom(Player): Room
+ CheckRoomExistsAt_Coordinate(): bool
+ DisplayCurrentRoom(Player)
Use
Use
Use